Session 13: NumPy Fundamentals

Data Science Mentorship Program (DSMP) 2022-23

1. Introduction to NumPy

What is NumPy?

NumPy stands for **Numerical Python**. It is the fundamental package for scientific computing in Python.

- **Core Purpose:** It is used for performing mathematical and logical operations on large arrays and matrices.
- **The Main Object:** The most important part of this library is the **NumPy Array Object** (also known as ndarray).
- **Other Objects:** It also provides objects like Masked Arrays and Matrices, along with a collection of routines for fast operations on arrays (including sorting, selecting, I/O, Fourier transforms, basic linear algebra, etc.).

Why NumPy over Python Lists?

A common question is: "Why do we need a separate array library when Python already has lists?"

1. **Speed (The Power of C):** Python is a "slow" language because it is interpreted. NumPy is written in **C**, which makes it incredibly fast. It uses a "wrapper" around C code so we can use simple Python syntax while getting C-level performance.
2. **Fixed Size:** Unlike Python lists (which can grow dynamically and occupy more memory), NumPy arrays have a fixed size at the time of creation.
3. **Homogeneous Data:** Every item in a NumPy array must be of the **same data type**. This allows the computer to store them in a continuous block in memory, making access much faster.
4. **Advanced Mathematics:** Performing complex math on millions of items is easy in NumPy (one line of code), whereas in Python lists, you would need complex loops and much more code.
5. **Industry Standard:** Almost all Data Science libraries like Pandas, Matplotlib, Scikit-learn, and TensorFlow are built on top of NumPy.

2. Creating NumPy Arrays

To use NumPy, you must first import it. We use the alias np by convention.

```
import numpy as np
```

A. Basic Creation (np.array)

1. 1D Array (Vector)

```
a = np.array([1, 2, 3])
print(a)
```

- **Line-by-line:**
  - np.array(): The function to create an array.
  - [1, 2, 3]: We pass a Python list as an argument.
  - Result: A 1D array, also called a Vector.

2. 2D Array (Matrix)

```
b = np.array([[1, 2, 3], [4, 5, 6]])
```

- **Line-by-line:**
  - We pass a "list of lists."
  - Each inner list represents a row.
  - Result: A 2D array with 2 rows and 3 columns.

3. 3D Array (Tensor)

```
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

- **Explanation:** This is a collection of 2D arrays. It is used for complex data like images (RGB channels).

4. Defining Data Type

```
d = np.array([1, 2, 3], dtype=float)
```

- **Explanation:** You can force the array to be of a specific type (e.g., float, int, complex, bool).

B. Functional Creation

1. np.arange

```
np.arange(1, 11, 2)
# Output: [1, 3, 5, 7, 9]
```

- **Explanation:** Works exactly like the Python range() function.

    - 1: Start (inclusive)
    - 11: Stop (exclusive)
    - 2: Step size

2. reshape

```
np.arange(1, 11).reshape(2, 5)
```

- **Explanation:** Changes the shape of the array.
    - **Rule:** The product of dimensions (2 * 5 = 10) must equal the number of items (10).

3. np.ones & np.zeros

```
np.ones((3, 4)) # Matrix of 3 rows, 4 columns filled with 1.0
np.zeros((3, 4)) # Matrix filled with 0.0
```

- **Why?** Used to "initialize" weights in Neural Networks or place holders for data.

4. np.random

```
np.random.random((3, 4))
```

- **Explanation:** Generates an array of the given shape with random values between 0 and 1.

5. np.linspace (Linear Space)

```
np.linspace(-10, 10, 10)
```

- **Explanation:** Generates 10 points between -10 and 10 that are **equidistant** (equal distance between every two points).

6. np.identity

```
np.identity(3)
```

- **Explanation:** Creates an Identity Matrix of 3x3 (1s on the diagonal, 0s elsewhere).

3. Important Attributes

Every NumPy array has specific properties (attributes):

1. **ndim**: Number of dimensions (1 for vector, 2 for matrix, 3 for tensor).
2. **shape**: Returns a tuple showing rows and columns.
    - *Tip:* In 3D (2, 2, 2), the first 2 means "two 2D arrays," and the remaining (2, 2) is the

shape of those 2D arrays.

3. **size**: Total number of items in the array.
4. **itemsize**: Memory occupied by a single item (in bytes).
    - *Note:* int32 takes 4 bytes, int64 takes 8 bytes.
5. **dtype**: Data type of the items (e.g., int64, float64).

4. Changing Data Types (astype)

You can convert an array's data type to save memory.

```
a3 = np.array([1, 2, 3]) # Default int64
a3.astype(np.int32)
```

- **Why?** If you store "Age" (max ~100), you don't need int64. Using int32 or even int8 can reduce a 10GB file to 8.5GB, saving significant memory.

5. Array Operations

A. Scalar Operations

Operations between a single number and an array.

```
a = np.array([1, 2, 3])
a + 2  # [3, 4, 5]
a * 2  # [2, 4, 6]
a ** 2 # [1, 4, 9]
a > 2  # [False, False, True] (Relational)
```

B. Vector Operations

Operations between two arrays.

```
a1 + a2 # Element-wise addition
```

6. Mathematical & Statistical Functions

A. Basic Math

- np.max() / np.min(): Find largest/smallest item.
- np.sum(): Sum of all items.
- np.prod(): Product of all items.
- **Axis Argument:** * axis=0: Operation performed **Column-wise**.

- axis=1: Operation performed **Row-wise**.

B. Statistics

- np.mean(): Average.
- np.median(): Middle value.
- np.std(): Standard Deviation.
- np.var(): Variance.

C. Dot Product

np.dot(matrix_a, matrix_b)

- **Rule:** Columns of A must equal Rows of B. This is the heart of Machine Learning calculations.

D. Trigonometry & Logs

- np.sin(), np.cos(), np.tan()
- np.log(), np.exp()
- np.round(), np.floor(), np.ceil()

7. Indexing & Slicing

A. Indexing

- **1D:** a[0] (same as Python list).
- **2D:** a[row_index, col_index]
  - *Example:* a[1, 2] fetches item in 2nd row, 3rd column.
- **3D:** a[tensor_index, row_index, col_index]

B. Slicing (Fetching multiple items)

Syntax: array[row_range, col_range]

Example 2D Slicing:

a2[0, :] # Fetch first row, all columns
a2[:, 2] # Fetch all rows, 3rd column
a2[1:, 1:3] # Fetch sub-matrix from 2nd row onwards and columns 2 to 3.

Example: "The Corners" of a Matrix:

To fetch 0, 3 (top corners) and 8, 11 (bottom corners) from a 3x4 matrix:

a2[::2, ::3]

- **Explanation:** ::2 means "start to end with step 2" (Rows 0 and 2). ::3 means "start to end with step 3" (Cols 0 and 3).

8. Reshaping Operations

1. **transpose (or .T)**: Swaps rows and columns.
2. **ravel**: Flattens any n-dimensional array into a 1D array.

9. Stacking & Splitting

A. Stacking (Joining)

- **np.hstack**: Horizontal stacking (side-by-side).
- **np.vstack**: Vertical stacking (one on top of another).
    - *Tip:* Shape must be compatible to stack!

B. Splitting (Breaking)

- **np.hsplit(array, parts)**: Breaks the array horizontally.
- **np.vsplit(array, parts)**: Breaks the array vertically.

Final Summary Checklist

- [x] **NumPy** is faster because it uses C and contiguous memory.
- [x] **np.array** is for basic creation; **arange**, **linspace** for sequence creation.
- [x] **Scalar ops** are element-wise; **Vector ops** are between arrays.
- [x] **Indexing** in 2D/3D requires [index, index].
- [x] **axis=0** is columns, **axis=1** is rows.
- [x] **hstack/vstack** are for joining data from different sources.

Class Conclusion: Today we covered the basics. Tomorrow we will discuss **Advanced NumPy** including **Broadcasting**, which is the "True Power" of NumPy.